MACHINE LEARNING LAB

EXPERIMENT -3

PROBLEM 1: PLAY TENNIS DATASET

1. Title

Implementation of the ID3 decision tree algorithm to predict whether to play tennis.

2. Aim

To build a decision tree using entropy and information gain, then classify a new weather sample.

3. Algorithm

1. Start the program.
2. Read the training dataset and class labels.
3. Calculate the entropy of the target class.
4. Calculate information gain for every attribute.
5. Select the attribute with the highest information gain as the decision node.
6. Repeat the splitting process recursively until the examples are classified.
7. Use the tree to classify a new sample and display the result.
8. Stop the program.

4. Python Code

```
import math
from pprint import pprint


def entropy(rows, target):
    counts = {}
    for row in rows:
        counts[row[target]] = counts.get(row[target], 0) + 1
    total = len(rows)
    return -sum((count / total) * math.log2(count / total)
                for count in counts.values())


def information_gain(rows, attribute, target):
```

```
    total_entropy = entropy(rows, target)
    values = list(dict.fromkeys(row[attribute] for row in rows))
    remainder = 0
    for value in values:
        subset = [row for row in rows if row[attribute] == value]
        remainder += (len(subset) / len(rows)) * entropy(subset, target)
    return total_entropy - remainder

def id3(rows, attributes, target):
    labels = [row[target] for row in rows]
    if len(set(labels)) == 1:
        return labels[0]
    if not attributes:
        return max(set(labels), key=labels.count)
    best = max(attributes, key=lambda a: information_gain(rows, a, target))
    tree = {best: {}}
    for value in dict.fromkeys(row[best] for row in rows):
        subset = [row for row in rows if row[best] == value]
        tree[best][value] = id3(
            subset, [a for a in attributes if a != best], target)
    return tree

def classify(tree, sample, default):
    if not isinstance(tree, dict):
        return tree
    attribute = next(iter(tree))
    value = sample.get(attribute)
    if value not in tree[attribute]:
        return default
    return classify(tree[attribute][value], sample, default)


data = [{'Outlook': 'Sunny', 'Humidity': 'High', 'Wind': 'Weak', 'Play': 'No'}, {'Outlook': 'Sunny',
'Humidity': 'High', 'Wind': 'Strong', 'Play': 'No'}, {'Outlook': 'Overcast', 'Humidity': 'High', 'Wind':
```

```
'Weak', 'Play': 'Yes'}, {'Outlook': 'Rain', 'Humidity': 'High', 'Wind': 'Weak', 'Play': 'Yes'}, {'Outlook':
'Rain', 'Humidity': 'Normal', 'Wind': 'Weak', 'Play': 'Yes'}, {'Outlook': 'Rain', 'Humidity': 'Normal',
'Wind': 'Strong', 'Play': 'No'}, {'Outlook': 'Overcast', 'Humidity': 'Normal', 'Wind': 'Strong', 'Play': 'Yes'},
{'Outlook': 'Sunny', 'Humidity': 'High', 'Wind': 'Weak', 'Play': 'No'}, {'Outlook': 'Sunny', 'Humidity':
'Normal', 'Wind': 'Weak', 'Play': 'Yes'}, {'Outlook': 'Rain', 'Humidity': 'Normal', 'Wind': 'Weak', 'Play':
'Yes'}, {'Outlook': 'Sunny', 'Humidity': 'Normal', 'Wind': 'Strong', 'Play': 'Yes'}, {'Outlook': 'Overcast',
'Humidity': 'High', 'Wind': 'Strong', 'Play': 'Yes'}]

target = 'Play'
attributes = [key for key in data[0] if key != target]
sample = {'Outlook': 'Sunny', 'Humidity': 'High', 'Wind': 'Strong'}

tree = id3(data, attributes, target)
default_class = max(set(row[target] for row in data),
                    key=[row[target] for row in data].count)
prediction = classify(tree, sample, default_class)

print("Decision Tree:")
pprint(tree)
print("New Sample:", sample)
print("Predicted Class:", prediction)
```

5. Input

Training dataset is included in the Python code.

New sample: Outlook = Sunny, Humidity = High, Wind = Strong

6. Output

Decision Tree: {'Outlook': {'Sunny': {'Humidity': {'High': 'No', 'Normal': 'Yes'}}, 'Overcast': 'Yes', 'Rain': {'Wind': {'Weak': 'Yes', 'Strong': 'No'}}}}

New Sample: {'Outlook': 'Sunny', 'Humidity': 'High', 'Wind': 'Strong'}

Predicted Class: No

7. Result

Thus, the ID3 algorithm was implemented successfully using the PLAY TENNIS DATASET dataset. The new sample was classified as No.

PROBLEM 2: STUDENT RESULT PREDICTION

1. Title

Implementation of ID3 to predict student pass/fail result.

2. Aim

To create a decision tree from student study, attendance, and assignment data and classify a new student.

3. Algorithm

1. Start the program.
2. Load student records with their result labels.
3. Find target entropy and information gain of each attribute.
4. Choose the attribute with maximum information gain.
5. Split the records and build subtrees recursively.
6. Classify the new student using the decision tree.
7. Display the tree and predicted result.

4. Python Code

```
import math
from pprint import pprint


def entropy(rows, target):
    counts = {}
    for row in rows:
        counts[row[target]] = counts.get(row[target], 0) + 1
    total = len(rows)
    return -sum((count / total) * math.log2(count / total)
                for count in counts.values())


def information_gain(rows, attribute, target):
    total_entropy = entropy(rows, target)
    values = list(dict.fromkeys(row[attribute] for row in rows))
    remainder = 0
    for value in values:
        subset = [row for row in rows if row[attribute] == value]
```

```
        remainder += (len(subset) / len(rows)) * entropy(subset, target)
    return total_entropy - remainder

def id3(rows, attributes, target):
    labels = [row[target] for row in rows]
    if len(set(labels)) == 1:
        return labels[0]
    if not attributes:
        return max(set(labels), key=labels.count)
    best = max(attributes, key=lambda a: information_gain(rows, a, target))
    tree = {best: {}}
    for value in dict.fromkeys(row[best] for row in rows):
        subset = [row for row in rows if row[best] == value]
        tree[best][value] = id3(
            subset, [a for a in attributes if a != best], target)
    return tree

def classify(tree, sample, default):
    if not isinstance(tree, dict):
        return tree
    attribute = next(iter(tree))
    value = sample.get(attribute)
    if value not in tree[attribute]:
        return default
    return classify(tree[attribute][value], sample, default)



data = [{'Attendance': 'Regular', 'Study': 'High', 'Assignment': 'Complete', 'Result': 'Pass'},
{'Attendance': 'Regular', 'Study': 'Low', 'Assignment': 'Complete', 'Result': 'Pass'}, {'Attendance':
'Irregular', 'Study': 'High', 'Assignment': 'Complete', 'Result': 'Pass'}, {'Attendance': 'Irregular', 'Study':
'Low', 'Assignment': 'Incomplete', 'Result': 'Fail'}, {'Attendance': 'Regular', 'Study': 'Low', 'Assignment':
'Incomplete', 'Result': 'Fail'}, {'Attendance': 'Irregular', 'Study': 'Low', 'Assignment': 'Complete', 'Result':
'Fail'}, {'Attendance': 'Regular', 'Study': 'High', 'Assignment': 'Incomplete', 'Result': 'Pass'},
{'Attendance': 'Irregular', 'Study': 'High', 'Assignment': 'Incomplete', 'Result': 'Pass'}]
```

```
target = 'Result'
attributes = [key for key in data[0] if key != target]
sample = {'Attendance': 'Regular', 'Study': 'Low', 'Assignment': 'Incomplete'}

tree = id3(data, attributes, target)
default_class = max(set(row[target] for row in data),
                    key=[row[target] for row in data].count)
prediction = classify(tree, sample, default_class)

print("Decision Tree:")
pprint(tree)
print("New Sample:", sample)
print("Predicted Class:", prediction)
```

5. Input

Training dataset is included in the Python code.

New sample: Attendance = Regular, Study = Low, Assignment = Incomplete

6. Output

Decision Tree: {'Study': {'High': 'Pass', 'Low': {'Attendance': {'Regular': {'Assignment': {'Complete': 'Pass', 'Incomplete': 'Fail'}}, 'Irregular': 'Fail'}}}}

New Sample: {'Attendance': 'Regular', 'Study': 'Low', 'Assignment': 'Incomplete'}

Predicted Class: Fail

7. Result

Thus, the ID3 algorithm was implemented successfully using the STUDENT RESULT PREDICTION dataset. The new sample was classified as Fail.

PROBLEM 3: BUYING A LAPTOP

1. Title

Implementation of ID3 to predict whether a customer will buy a laptop.

2. Aim

To build a decision tree from laptop price, weight, and battery information and classify a new customer.

3. Algorithm

1. Start the program.
2. Prepare the laptop purchasing dataset.
3. Calculate entropy for the Buy class.
4. Calculate information gain for Price, Weight, and Battery.
5. Select the attribute with the highest information gain.
6. Continue splitting until leaf nodes give the class labels.
7. Classify a new laptop preference sample and display the output.

4. Python Code

```
import math
from pprint import pprint


def entropy(rows, target):
    counts = {}
    for row in rows:
        counts[row[target]] = counts.get(row[target], 0) + 1
    total = len(rows)
    return -sum((count / total) * math.log2(count / total)
                for count in counts.values())


def information_gain(rows, attribute, target):
    total_entropy = entropy(rows, target)
    values = list(dict.fromkeys(row[attribute] for row in rows))
    remainder = 0
    for value in values:
        subset = [row for row in rows if row[attribute] == value]
```

```
        remainder += (len(subset) / len(rows)) * entropy(subset, target)
    return total_entropy - remainder

def id3(rows, attributes, target):
    labels = [row[target] for row in rows]
    if len(set(labels)) == 1:
        return labels[0]
    if not attributes:
        return max(set(labels), key=labels.count)
    best = max(attributes, key=lambda a: information_gain(rows, a, target))
    tree = {best: {}}
    for value in dict.fromkeys(row[best] for row in rows):
        subset = [row for row in rows if row[best] == value]
        tree[best][value] = id3(
            subset, [a for a in attributes if a != best], target)
    return tree

def classify(tree, sample, default):
    if not isinstance(tree, dict):
        return tree
    attribute = next(iter(tree))
    value = sample.get(attribute)
    if value not in tree[attribute]:
        return default
    return classify(tree[attribute][value], sample, default)



data = [{'Price': 'High', 'Weight': 'Light', 'Battery': 'Long', 'Buy': 'Yes'}, {'Price': 'High', 'Weight':
'Heavy', 'Battery': 'Long', 'Buy': 'Yes'}, {'Price': 'Low', 'Weight': 'Light', 'Battery': 'Short', 'Buy': 'No'},
{'Price': 'Medium', 'Weight': 'Light', 'Battery': 'Long', 'Buy': 'Yes'}, {'Price': 'Low', 'Weight': 'Heavy',
'Battery': 'Short', 'Buy': 'No'}, {'Price': 'Medium', 'Weight': 'Heavy', 'Battery': 'Long', 'Buy': 'Yes'},
{'Price': 'High', 'Weight': 'Light', 'Battery': 'Short', 'Buy': 'Yes'}, {'Price': 'Low', 'Weight': 'Light',
'Battery': 'Long', 'Buy': 'No'}]
```

```
target = 'Buy'
attributes = [key for key in data[0] if key != target]
sample = {'Price': 'Low', 'Weight': 'Light', 'Battery': 'Long'}

tree = id3(data, attributes, target)
default_class = max(set(row[target] for row in data),
                    key=[row[target] for row in data].count)
prediction = classify(tree, sample, default_class)

print("Decision Tree:")
pprint(tree)
print("New Sample:", sample)
print("Predicted Class:", prediction)
```

5. Input

Training dataset is included in the Python code.

New sample: Price = Low, Weight = Light, Battery = Long

6. Output

Decision Tree: {'Price': {'High': 'Yes', 'Low': 'No', 'Medium': 'Yes'}}

New Sample: {'Price': 'Low', 'Weight': 'Light', 'Battery': 'Long'}

Predicted Class: No

7. Result

Thus, the ID3 algorithm was implemented successfully using the BUYING A LAPTOP dataset. The new sample was classified as No.

PROBLEM 4: LOAN APPROVAL

1. Title

Implementation of ID3 to predict loan approval from customer details.

2. Aim

To build a decision tree using income, credit history, and job stability, then classify a loan application.

3. Algorithm

1. Start the program.
2. Load the customer dataset with Loan labels.
3. Calculate the entropy of the target class.
4. Compute information gain for each customer attribute.
5. Choose the best attribute and create branches for its values.
6. Repeat until each branch reaches a class label or no attributes remain.
7. Classify the new loan application and display the result.

4. Python Code

```
import math
from pprint import pprint


def entropy(rows, target):
    counts = {}
    for row in rows:
        counts[row[target]] = counts.get(row[target], 0) + 1
    total = len(rows)
    return -sum((count / total) * math.log2(count / total)
                for count in counts.values())


def information_gain(rows, attribute, target):
    total_entropy = entropy(rows, target)
    values = list(dict.fromkeys(row[attribute] for row in rows))
    remainder = 0
    for value in values:
        subset = [row for row in rows if row[attribute] == value]
```

```
        remainder += (len(subset) / len(rows)) * entropy(subset, target)
    return total_entropy - remainder

def id3(rows, attributes, target):
    labels = [row[target] for row in rows]
    if len(set(labels)) == 1:
        return labels[0]
    if not attributes:
        return max(set(labels), key=labels.count)
    best = max(attributes, key=lambda a: information_gain(rows, a, target))
    tree = {best: {}}
    for value in dict.fromkeys(row[best] for row in rows):
        subset = [row for row in rows if row[best] == value]
        tree[best][value] = id3(
            subset, [a for a in attributes if a != best], target)
    return tree

def classify(tree, sample, default):
    if not isinstance(tree, dict):
        return tree
    attribute = next(iter(tree))
    value = sample.get(attribute)
    if value not in tree[attribute]:
        return default
    return classify(tree[attribute][value], sample, default)



data = [{'Income': 'High', 'Credit': 'Good', 'Job': 'Stable', 'Loan': 'Yes'}, {'Income': 'High', 'Credit': 'Poor',
'Job': 'Stable', 'Loan': 'No'}, {'Income': 'Medium', 'Credit': 'Good', 'Job': 'Stable', 'Loan': 'Yes'},
{'Income': 'Low', 'Credit': 'Good', 'Job': 'Unstable', 'Loan': 'No'}, {'Income': 'Low', 'Credit': 'Poor', 'Job':
'Stable', 'Loan': 'No'}, {'Income': 'Medium', 'Credit': 'Poor', 'Job': 'Unstable', 'Loan': 'No'}, {'Income':
'High', 'Credit': 'Good', 'Job': 'Unstable', 'Loan': 'Yes'}, {'Income': 'Medium', 'Credit': 'Good', 'Job':
'Unstable', 'Loan': 'Yes'}]
```

```
target = 'Loan'
attributes = [key for key in data[0] if key != target]
sample = {'Income': 'Medium', 'Credit': 'Good', 'Job': 'Stable'}

tree = id3(data, attributes, target)
default_class = max(set(row[target] for row in data),
                    key=[row[target] for row in data].count)
prediction = classify(tree, sample, default_class)

print("Decision Tree:")
pprint(tree)
print("New Sample:", sample)
print("Predicted Class:", prediction)
```

5. Input

Training dataset is included in the Python code.

New sample: Income = Medium, Credit = Good, Job = Stable

6. Output

Decision Tree: {'Credit': {'Good': {'Income': {'High': 'Yes', 'Medium': 'Yes', 'Low': 'No'}}, 'Poor': 'No'}}

New Sample: {'Income': 'Medium', 'Credit': 'Good', 'Job': 'Stable'}

Predicted Class: Yes

7. Result

Thus, the ID3 algorithm was implemented successfully using the LOAN APPROVAL dataset. The new sample was classified as Yes.

PROBLEM 5: PLAY CRICKET

1. Title

Implementation of ID3 to decide whether a person can play cricket.

2. Aim

To construct a decision tree using weather, day, and wind conditions and classify a new sample.

3. Algorithm

1. Start the program.
2. Read the cricket dataset and target labels.
3. Calculate the entropy of Play.
4. Calculate information gain for each attribute.
5. Select the attribute with the highest information gain as the root.
6. Build branches recursively using the remaining attributes.
7. Predict the class of a new cricket sample and display the result.

4. Python Code

```
import math
from pprint import pprint


def entropy(rows, target):
    counts = {}
    for row in rows:
        counts[row[target]] = counts.get(row[target], 0) + 1
    total = len(rows)
    return -sum((count / total) * math.log2(count / total)
                for count in counts.values())


def information_gain(rows, attribute, target):
    total_entropy = entropy(rows, target)
    values = list(dict.fromkeys(row[attribute] for row in rows))
    remainder = 0
    for value in values:
        subset = [row for row in rows if row[attribute] == value]
```

```
        remainder += (len(subset) / len(rows)) * entropy(subset, target)
    return total_entropy - remainder

def id3(rows, attributes, target):
    labels = [row[target] for row in rows]
    if len(set(labels)) == 1:
        return labels[0]
    if not attributes:
        return max(set(labels), key=labels.count)
    best = max(attributes, key=lambda a: information_gain(rows, a, target))
    tree = {best: {}}
    for value in dict.fromkeys(row[best] for row in rows):
        subset = [row for row in rows if row[best] == value]
        tree[best][value] = id3(
            subset, [a for a in attributes if a != best], target)
    return tree

def classify(tree, sample, default):
    if not isinstance(tree, dict):
        return tree
    attribute = next(iter(tree))
    value = sample.get(attribute)
    if value not in tree[attribute]:
        return default
    return classify(tree[attribute][value], sample, default)



data = [{'Weather': 'Sunny', 'Day': 'Weekend', 'Wind': 'Low', 'Play': 'Yes'}, {'Weather': 'Sunny', 'Day':
'Weekday', 'Wind': 'Low', 'Play': 'Yes'}, {'Weather': 'Rainy', 'Day': 'Weekend', 'Wind': 'High', 'Play':
'No'}, {'Weather': 'Cloudy', 'Day': 'Weekend', 'Wind': 'Low', 'Play': 'Yes'}, {'Weather': 'Rainy', 'Day':
'Weekday', 'Wind': 'High', 'Play': 'No'}, {'Weather': 'Cloudy', 'Day': 'Weekday', 'Wind': 'High', 'Play':
'Yes'}, {'Weather': 'Sunny', 'Day': 'Weekend', 'Wind': 'High', 'Play': 'Yes'}, {'Weather': 'Rainy', 'Day':
'Weekend', 'Wind': 'Low', 'Play': 'No'}]
```

```
target = 'Play'
attributes = [key for key in data[0] if key != target]
sample = {'Weather': 'Rainy', 'Day': 'Weekend', 'Wind': 'Low'}

tree = id3(data, attributes, target)
default_class = max(set(row[target] for row in data),
                    key=[row[target] for row in data].count)
prediction = classify(tree, sample, default_class)

print("Decision Tree:")
pprint(tree)
print("New Sample:", sample)
print("Predicted Class:", prediction)
```

5. Input

Training dataset is included in the Python code.

New sample: Weather = Rainy, Day = Weekend, Wind = Low

6. Output

Decision Tree: {'Weather': {'Sunny': 'Yes', 'Rainy': 'No', 'Cloudy': 'Yes'}}

New Sample: {'Weather': 'Rainy', 'Day': 'Weekend', 'Wind': 'Low'}

Predicted Class: No

7. Result

Thus, the ID3 algorithm was implemented successfully using the PLAY CRICKET dataset. The new sample was classified as No.